

Full EFX Allocations Exist for Four Agents and Eight Goods with Additive Valuations

Eduard Tykhoniuk

June 2026

Abstract

The existence of *envy-free up to any good* (EFX) allocations is one of the central open problems in the fair division of indivisible goods. It is known to hold for two agents with general valuations and for three agents with additive valuations; for four agents it is known to hold *almost* fully — an EFX allocation always exists if one item may be left unallocated. Whether a *full* (complete) EFX allocation always exists for four agents remains open. In this report we settle the question *for a fixed instance size*: we show that every **additive** profile with four agents and eight goods admits a full EFX allocation. The argument reduces the existence statement to a single satisfiability query over linear real arithmetic: the negation of EFX existence is satisfiable if and only if a counterexample profile exists. We encode this query with the Z3 SMT solver and exploit a goods-relabelling symmetry that, as we verify empirically, is needed for the larger instances to finish; after roughly 62.5 hours the solver returns **unsat**. To the best of our knowledge this is the first verification of full EFX existence for four agents and eight goods under additive valuations.

Contents

1	Introduction	2
2	Preliminaries	3
3	Reducing existence to satisfiability	4
3.1	Negating the existence statement	4
3.2	Deciding the query: linear real arithmetic	4
3.3	Encoding a single allocation: the nonenvy predicate	5
3.4	Why restricting to non-empty partitions is conservative	5
3.5	Symmetry reduction	6
3.6	Counting the candidate allocations	6
3.7	The shape of the search space	6
4	Implementation	7
4.1	Z3 pipeline	7
4.2	The relabelling symmetry break, measured	8
5	Results	8
6	Extension to non-negative valuations	8
7	Related work and novelty	9

8	Discussion and limitations	11
9	Repository structure and running the code	11
10	Conclusion and future work	12
	References	12
A	Additional material	14

1 Introduction

Fair division studies how to partition a set of resources among agents with possibly conflicting preferences so that the outcome is, in some precise sense, fair. When the resources are *indivisible goods* — items that cannot be split, such as houses, courses, tasks, or shifts — the classical benchmark of *envy-freeness* (no agent prefers another agent’s bundle to its own) is generally unachievable: with one good and two agents, whoever leaves empty-handed envies the other. Research has therefore concentrated on *relaxations* of envy-freeness that are achievable yet still meaningful.

The most prominent relaxations are *envy-free up to one good* (EF1) and *envy-free up to any good* (EFX). Under EF1 an agent may envy another, provided the envy disappears once *some* single good is removed from the envied bundle; EF1 allocations always exist and can be computed in polynomial time, even for general monotone valuations [7]; see also [4]. EFX is the strictly stronger requirement that the envy disappears once *any* single good is removed [5, 10]. EFX is one of the strongest such fairness notions for indivisible goods, and whether an EFX allocation always exists is a major open problem in the area [2].

State of the art. EFX existence is known in only a handful of regimes. It holds for two agents with arbitrary monotone valuations [10], and for three agents with additive valuations [6]. It also holds, for general monotone valuations and any number of agents, whenever the number of goods is small relative to the number of agents: Mahara [8] proved that an EFX allocation always exists when $M \leq N + 3$. For $N = 4$ this already settles all sizes up to $M = 7$, so the smallest open four-agent case is $M = N + 4 = 8$. For four agents, Berger, Cohen, Feldman and Fiat [3] proved that an EFX allocation always exists *if at most one good is left unallocated* — an “almost full” guarantee — and, more generally, that with N additive agents an EFX allocation exists leaving at most $N - 2$ goods unallocated. Whether a *full* EFX allocation (one that allocates *all* goods) always exists for four agents is, to date, open even for additive valuations. On the negative side, Akrami, Mayorov, Mehlhorn, Srinivas and Weidenbach [1] very recently produced the first counterexamples to EFX existence under *submodular* (and hence monotone) valuations, showing that for $n \geq 3$ agents EFX need not exist once $m \geq n + 5$; they leave the *additive* case wide open.

Beyond exact existence, a large body of work studies what can be guaranteed when exactness is relaxed: approximate EFX (where an agent’s value for its own bundle is within a multiplicative factor of the envied bundle minus a good), EFX with bounded charity (a small number of unallocated goods, as above), restricted valuation classes (binary, two-valued, or few distinct types), and EFX on graphs where agents only compare with neighbours. The survey of Amanatidis et al. [2] gives a comprehensive map of these directions and frames exact EFX existence for general additive valuations as a central open question.

This report. We approach full EFX existence *computationally*, for one fixed instance size. Concretely, we prove:

Theorem 1 (Main result). *Every instance with four agents, eight goods and non-negative additive valuations admits a full EFX allocation.*

The proof is computational. The statement “EFX always exists for a fixed (N, M) ” has the form: *for every* valuation profile *there exists* an allocation that is EFX. Its negation reads: *there exists* a profile such that *every* allocation fails EFX. Because the valuations are real-valued and the EFX conditions are linear inequalities, this negation is a single existential query in linear real arithmetic, which an SMT solver decides exactly: a satisfying assignment is a counterexample profile, while unsatisfiability means no counterexample exists, i.e. EFX always exists. Running the query for $N = 4$, $M = 8$ returns `unsat`, which establishes Theorem 1 for strictly positive valuations; a short reduction (Section 6) then removes the positivity assumption, extending it to all non-negative profiles in the strong EFX_0 sense.

The remainder of the report sets up this reduction (Section 3), describes the Z3-based implementation and a goods-relabelling symmetry that we find empirically to be needed for the larger instances to finish (Section 4), reports the result (Section 5), and places it against the literature (Section 7).

Repository. The code, report sources, logs, and retained proof artifacts are available at <https://github.com/MAKMED1337/EFX>.

2 Preliminaries

Instances. There are N agents $[N] = \{1, \dots, N\}$ and M indivisible goods $[M] = \{1, \dots, M\}$. Each agent i has an *additive* valuation $v_i: 2^{[M]} \rightarrow \mathbb{R}_{\geq 0}$, meaning $v_i(S) = \sum_{g \in S} v_i(\{g\})$ for every bundle $S \subseteq [M]$. We write $w_{i,g} := v_i(\{g\})$ for the value agent i assigns to good g , so a profile is an $N \times M$ matrix $(w_{i,g})$. Throughout we assume valuations are *strictly positive*, $w_{i,g} > 0$; this is the standard “goods” setting and the one our solver encodes.

Allocations. An *allocation* is an ordered partition $X = (X_1, \dots, X_N)$ of a subset of $[M]$ into pairwise-disjoint bundles, where X_i is the bundle given to agent i . The allocation is *full* (or *complete*) if $\bigcup_i X_i = [M]$, i.e. no good is left over.

Definition 1 (Envy notions). Fix valuations $v_1, \dots, v_N$ and an allocation X . In each clause below, i and j range over all ordered pairs of agents.

- X is *envy-free* (EF) if

$$v_i(X_i) \geq v_i(X_j).$$

- X is *envy-free up to one good* (EF1) if

$$\exists g \in X_j : v_i(X_i) \geq v_i(X_j \setminus \{g\})$$

(and is defined to hold when $X_j = \emptyset$).

- X is *envy-free up to any good* (EFX) if

$$\forall g \in X_j : v_i(X_i) \geq v_i(X_j \setminus \{g\}).$$

The only difference between EF1 and EFX is the quantifier on g ($\exists$ versus $\forall$): EF1 asks that *some* good can be removed to eliminate the envy, EFX that *every* good does.

Equivalently, X is EFX iff for every ordered pair (i, j) ,

$$v_i(X_i) \geq v_i(X_j) - \min_{g \in X_j} v_i(\{g\}),$$

because under additive (positive) valuations the bundle $X_j \setminus \{g\}$ of largest value to i is the one obtained by removing i ’s *least*-valued good in X_j . When $X_j = \emptyset$ there is no good to remove and the condition holds trivially.

A note on the literature. Two variants of EFX appear in the literature [2], differing in how they treat a good the envious agent values at zero. EFX_+ , as originally defined by Caragiannis et al. [5], removes only *positively*-valued goods ($g \in X_j$ with $v_i(\{g\}) > 0$); the stronger EFX_0 , used by Plaut and Roughgarden [10], removes *any* good $g \in X_j$, including one valued at zero. The corresponding existence guarantees are equivalent at any fixed instance size (Corollary 2), so the choice between them does not affect our results. We adopt the stronger EFX_0 notion and state all of our results for it; the encoding of Section 3 tests precisely this condition.

The existence question we study is:

For fixed N and M , does every additive profile admit a full EFX allocation?

A *counterexample* is a profile for which *no* full EFX allocation exists. Theorem 1 asserts that for $(N, M) = (4, 8)$ there is no counterexample.

3 Reducing existence to satisfiability

3.1 Negating the existence statement

Fix (N, M) and let $\mathcal{P}$ denote the (finite) set of ordered partitions of $[M]$ into N non-empty bundles. The statement we want to prove is

$$\underbrace{\forall (w_{i,g}) > 0}_{\text{every profile}} \quad \underbrace{\exists P \in \mathcal{P}}_{\text{some allocation}} \quad \text{EFX}(P, w),$$

where $\text{EFX}(P, w)$ is the predicate “ P is EFX under profile w ”. There are infinitely many profiles, so the statement cannot be decided by checking them one by one. Its negation, however, is purely existential in w ,

$$\exists (w_{i,g}) > 0 \quad \bigwedge_{P \in \mathcal{P}} \neg \text{EFX}(P, w). \quad (\star)$$

That is: a counterexample is a single profile for which every allocation fails EFX. Statement $(\star)$ is a conjunction of disjunctions of *linear* inequalities in the real variables $w_{i,g}$ — the fragment *linear real arithmetic* (LRA) that SMT solvers decide exactly.¹ Feeding $(\star)$ to such a solver yields:

- **sat** $\Rightarrow$ a counterexample profile, which can be read off as a rational matrix; or
- **unsat** $\Rightarrow (\star)$ has no solution, hence the original statement holds: EFX always exists.

3.2 Deciding the query: linear real arithmetic

Why can a solver decide $(\star)$ *exactly*, rather than only search heuristically? The formula $(\star)$ lives in *quantifier-free linear real arithmetic* (QF-LRA): a Boolean combination of linear inequalities $\sum_g a_g w_{i,g} \bowtie c$ with rational coefficients, where $\bowtie \in \{<, \leq, =, \geq, >\}$. This theory is *decidable*: a **conjunction** of linear inequalities (an intersection of half-spaces) over the reals is satisfiable iff the corresponding polyhedron is non-empty, which a simplex-style procedure settles in finitely many exact-rational pivots; an SMT solver lifts this to arbitrary Boolean structure by searching over truth assignments to the inequalities in a SAT-solving loop (the DPLL(T) procedure underlying SMT). All arithmetic is carried out over the rationals $\mathbb{Q}$, with no floating-point approximation and no ε -tolerances, so an **unsat** verdict is a true impossibility rather than a numerical artefact — provided the solver reasons exactly. Z3’s QF-LRA core is designed to do so, using exact

¹Linearity also bounds the search a priori: if $(\star)$ is satisfiable then it has a *rational* solution, because a feasible system of linear inequalities with rational coefficients has a rational point (the feasible region of the strict system is open and non-empty, hence contains a rational point). A counterexample profile, if one exists, may therefore be taken rational, matching the exact rational arithmetic the solver uses (Section 3.2).

arbitrary-precision rational arithmetic. This exactness is what lets us treat valuations as arbitrary positive reals: proving **unsat** over $\mathbb{R}$ rules out *all* real — in particular all rational and all integer — profiles at once.

3.3 Encoding a single allocation: the nonenvy predicate

For a fixed partition $P = (P_1, \dots, P_N)$, the predicate $\text{EFX}(P, w)$ expands into one inequality per (envious agent, envied agent, removed good) triple. For agent i looking at agent j 's bundle P_j , and for each good $k \in P_j$, **EFX** demands

$$\sum_{g \in P_i} w_{i,g} \geq \sum_{g \in P_j} w_{i,g} - w_{i,k}. \quad (1)$$

$\text{EFX}(P, w)$ is exactly the conjunction of (1) over all $i \neq j$ and all $k \in P_j$. Two simplifications are sound and are used in the implementation:

- **An envied bundle that is a singleton imposes no constraint:** if $|P_j| \leq 1$, the terms from j may be dropped (Observation 1).
- **Own bundle is excluded** ($i = j$ is skipped), since an agent never envies itself.

Observation 1 (Singletons impose no **EFX** constraint). *Under non-negative valuations, if $|P_j| \leq 1$ then no agent $i \neq j$ can **EFX-envy** j , so the corresponding constraints (1) always hold and can be dropped.*

Proof. If $P_j = \emptyset$ there is no good k to quantify over and the predicate is empty. If $P_j = \{k\}$, the only instance of (1) reads $\sum_{g \in P_i} w_{i,g} \geq w_{i,k} - w_{i,k} = 0$, which holds because all valuations are non-negative. In both cases the singleton adds no real constraint. $\square$

Example expansion. In general a partition contributes one inequality for every ordered pair (i, j) with $|P_j| \geq 2$ and every good $k \in P_j$. For $N = 2$, $M = 3$ and the partition $P = (\{1\}, \{2, 3\})$ only the pair $(1, 2)$ qualifies — agent 2 faces the singleton $P_1 = \{1\}$, which contributes nothing by Observation 1 — so the block is small:

$$\underbrace{w_{1,1} \geq (w_{1,2} + w_{1,3}) - w_{1,2}}_{\text{remove good 2}} \quad \wedge \quad \underbrace{w_{1,1} \geq (w_{1,2} + w_{1,3}) - w_{1,3}}_{\text{remove good 3}},$$

i.e. $w_{1,1} \geq w_{1,3}$ and $w_{1,1} \geq w_{1,2}$. Each partition yields such a block, and the solver asserts the negation of their disjunction: that no partition's block holds in full.

3.4 Why restricting to non-empty partitions is conservative

The set $\mathcal{P}$ ranges only over partitions in which *every* agent receives at least one good. **EFX** allocations are not required to be of this form, so $\mathcal{P}$ omits some valid candidate allocations (those that hand an agent the empty bundle). It is important to see that this restriction is *safe for the existence direction* we care about.

Proposition 1 (Soundness of the restricted search). *Let $\mathcal{P}' \subseteq \mathcal{P}^{\text{all}}$ be any subset of the full set of allocations. If the query $\exists w > 0 : \bigwedge_{P \in \mathcal{P}'} \neg \text{EFX}(P, w)$ is unsatisfiable (has no solution w , i.e. no counterexample), then **EFX** always exists (even when ranging over $\mathcal{P}^{\text{all}}$).*

Proof. Removing allocations from the conjunction in $(\star)$ can only *weaken* it: $\bigwedge_{P \in \mathcal{P}^{\text{all}}} \neg \text{EFX} \implies \bigwedge_{P \in \mathcal{P}'} \neg \text{EFX}$. Contrapositively, if the smaller conjunction over $\mathcal{P}'$ is unsatisfiable, so is the larger one over $\mathcal{P}^{\text{all}}$; hence no counterexample profile exists and **EFX** always exists. $\square$

Restricting the candidate set can therefore only introduce false counterexamples, never false claims of existence; an **unsat** on the restricted set $\mathcal{P}$ still proves existence. This is what our run establishes for $(N, M) = (4, 8)$.²

3.5 Symmetry reduction

Relabelling the goods is a symmetry of the problem: permuting the columns of a profile w and applying the same permutation to every bundle preserves EFX. We break this symmetry by fixing one agent’s view. Without loss of generality the goods are indexed so that agent 1’s valuations are non-decreasing,

$$w_{1,1} \leq w_{1,2} \leq \dots \leq w_{1,M}.$$

This is encoded as $M - 1$ extra inequalities. It is sound because every profile is equivalent, after relabelling goods, to one in sorted form, and it removes a factor of up to $M!$ from the search space (for $M = 8$, up to 40320). The realised speedup can be smaller than $M!$, since Z3 already performs some symmetry reduction internally; Section 4.2 measures the actual effect.

3.6 Counting the candidate allocations

The number of ordered partitions of M goods into N non-empty labelled bundles is the surjection count $N! \left\{ \begin{smallmatrix} M \\ N \end{smallmatrix} \right\}$, where $\left\{ \begin{smallmatrix} M \\ N \end{smallmatrix} \right\}$ is a Stirling number of the second kind. For the three sizes we ran ($N = 4$, $M \in \{6, 7, 8\}$):

$$4! \cdot \left\{ \begin{smallmatrix} 6 \\ 4 \end{smallmatrix} \right\} = 1560, \quad 4! \cdot \left\{ \begin{smallmatrix} 7 \\ 4 \end{smallmatrix} \right\} = 8400, \quad 4! \cdot \left\{ \begin{smallmatrix} 8 \\ 4 \end{smallmatrix} \right\} = 40824.$$

Each of these 40824 partitions contributes its block of inequalities (1); the solver must refute the conjunction of all their negations simultaneously.

3.7 The shape of the search space

We now explain why the problem is hard, which accounts for both the runtime and the limits of the approach. The formula $(\star)$ has only 32 real variables, but a large Boolean structure: a counterexample profile must violate at least one EFX inequality of each of the 40824 allocations at once. The solver does not go through profiles or combinations one by one. Instead it uses the linear structure to prune: once a partial set of constraints has no solution, every larger set that contains it has no solution either, so the solver can skip them all.

Two consequences follow. First, the difficulty is highly sensitive to M : going from $M = 6$ to $M = 8$ multiplies the allocation count by about 26 and lengthens every constraint, and the running time grows from under a second to about 62 hours (Section 5). Second, this sensitivity also means the $(4, 8)$ run is computationally demanding: a related three-agent / seven-good EFX query did not terminate within a 40-hour budget for the Z3-over-LRA encoding of [1], so termination at this scale cannot be taken for granted. The two encodings are not directly comparable — theirs is over *monotone* valuations (a real variable per bundle, with several thousand constraints), whereas ours is *additive* (the 32 per-good variables of $(\star)$) — so their non-termination is suggestive of the difficulty rather than a prediction for our search space; but both run Z3 over QF_LRA at a comparable allocation count, which is enough to make the point.

²In any case, for positive additive valuations with $M \geq N$ an allocation that leaves an agent empty is never EFX unless every other bundle is a singleton, so the restriction discards little here; Proposition 1 makes the argument independent of this.

4 Implementation

4.1 Z3 pipeline

The main prover is a C++ program built against the Z3 C++ API [9]. It (i) creates the $N \times M$ real variables $w_{i,g}$ and asserts positivity and the sortedness constraints of Section 3.5; (ii) enumerates the non-empty ordered partitions; (iii) builds, for each partition, the conjunction **nonenvy** of constraints (1); (iv) asserts the negation of their disjunction; and (v) calls the solver. The core of the encoding is shown below. The instance size is supplied on the command line as $N\ M$.

Listing 1: The **nonenvy** predicate (constraints (1)), from `main.cpp`.

```

1  z3::expr nonenvy(z3::context &ctx,
2      int N,
3      const std::vector<std::vector<int>> &partition,
4      const std::vector<std::vector<z3::expr>> &w) {
5      z3::expr_vector constraints(ctx);
6      for (int i = 0; i < N; ++i) {
7          z3::expr total = ctx.real_val(0);          // value of i's own bundle
8          for (int j : partition[i]) total = total + w[i][j];
9
10         for (int j = 0; j < N; ++j) {
11             if (i == j) continue;
12             if (partition[j].size() <= 1) continue; // singletons: EFX is automatic
13             z3::expr other = ctx.real_val(0);       // value of j's bundle to i
14             for (int k : partition[j]) other = other + w[i][k];
15             // For every k in j's bundle: i must not envy j after removing k.
16             for (int k : partition[j])
17                 constraints.push_back(total >= other - w[i][k]);
18         }
19     }
20     return z3::mk_and(constraints);
21 }

```

Listing 2: Assembling and refuting the global disjunction, from `main.cpp`.

```

1  int N = std::atoi(argv[1]);          // number of agents
2  int M = std::atoi(argv[2]);          // number of goods
3
4  // First agent's valuations sorted (goods-relabelling symmetry break).
5  for (int j = 0; j < M - 1; ++j) solver.add(w[0][j] <= w[0][j + 1]);
6
7  auto partitions = ordered_groups(N, M); // 40824 for (4,8)
8  z3::expr_vector valid_exprs(ctx);
9  for (const auto &part : partitions)
10     valid_exprs.push_back(nonenvy(ctx, N, part, w));
11  z3::expr or_valid = z3::mk_or(valid_exprs);
12  solver.add(!or_valid);                // assert (*) : no allocation is EFX
13
14  z3::check_result result = solver.check(); // unsat => EFX always exists

```

The program also exports the assembled query in the standard SMT-LIB 2 format so that it can be replayed by any compliant solver; for $(N, M) = (4, 8)$ this script (`problem_4_8.smt2`) is about 9 MiB, compared with 0.24 MiB for $(4, 6)$. Its structure is a flat declaration of the 32 real variables, the positivity and sortedness assertions, and one large negated disjunction:

Listing 3: Shape of the exported SMT-LIB 2 query (abridged).

```

1  (declare-fun w_0_0 () Real) ... (declare-fun w_3_7 () Real)
2  (assert (> w_0_0 0)) ... ; positivity, 32 of these
3  (assert (<= w_0_0 w_0_1)) ... ; agent 0 sorted, 7 of these
4  (assert (not (or
5      (and (>= ...) (>= ...) ...) ; nonenvy(partition 1)

```

```

6   ...
7   (and (>= ...) (>= ...) ...)))           ; nonenvy(partition 40824)
8 (check-sat)

```

4.2 The relabelling symmetry break, measured

Section 3.5 argued that fixing agent 1’s valuations in non-decreasing order removes a factor of up to $M!$ from the search. To measure its effect we ran the prover at $N = 4$ for $M \in \{6, 7\}$ both with and without the three optimizations, leaving everything else identical; Table 1 reports the wall-clock.

M	#partitions	with relabelling	without relabelling
6	1,560	< 1 s	8 s
7	8,400	196 s (≈ 3.3 min)	> 30 min (did not finish)

Table 1: Effect of the agent-1 relabelling symmetry break ($N = 4$). Disabling it leaves the $M!$ permutation copies of every profile in the search, and the solver must rediscover the same refutation for each.

The symmetry break helps, and more so as M grows. At $M = 7$ the unsorted run did not finish within a 30-minute cap while the sorted run took about 3 minutes — at least a tenfold reduction, and only a lower bound since the unsorted run did not terminate. We did not run the unsorted $M = 8$ case.

5 Results

For $(N, M) = (4, 8)$ the solver returned **unsat** (“No counterexample”), establishing Theorem 1 for strictly positive valuations; Section 6 lifts it to all non-negative profiles. The smaller instances $(4, 6)$ and $(4, 7)$ were solved first and likewise admit a full EFX allocation for every profile. These two sizes satisfy $M \leq N + 3$ and are therefore already covered by Mahara’s analytic existence theorem [8]; we include them as benchmarks — a check that the pipeline reproduces known results — and only $(4, 8) = (N, N + 4)$ is new. Table 2 summarises the runs.

Instance (N, M)	#partitions	Wall-clock	Outcome
$(4, 6)$	1,560	< 1 s	unsat (full EFX exists)
$(4, 7)$	8,400	196 s	unsat (full EFX exists)
$(4, 8)$	40,824	62 h 34 m	unsat (full EFX exists)

Table 2: Outcomes of the SMT runs; peak memory for the $(4, 8)$ run was about 415 MB.

Observation 2. *For this instance size, Theorem 1 is stronger than the “almost full” guarantee of [3]: it shows that the single item that their result may leave unallocated is never needed at $(4, 8)$ — a complete allocation always exists.*

6 Extension to non-negative valuations

The solver assumes strictly positive valuations, but the result extends to all *non-negative* additive profiles, including those with zero entries, and the argument stays within the additive class.

Theorem 2. *Fix N, M . If every strictly positive additive profile admits a full EFX allocation over the ordered partitions, then every non-negative additive profile admits a full EFX_0 allocation over the same family.*

Proof. The argument perturbs the zero entries to obtain a strictly positive profile, applies the hypothesis to get a full EFX allocation there (on a strictly positive profile no good is valued at zero, so this allocation is already EFX_0), and shows the same allocation stays EFX_0 for the original profile.

Each agent's EFX_0 constraints involve only that agent's own weights, so we argue one agent i at a time, with weights $w_{i,\cdot} \geq 0$. If all of agent i 's weights are zero, every EFX_0 constraint for i reads $0 \geq 0$ and holds; otherwise i has a positive weight. Let $W_i = \{\sum_{g \in S} w_{i,g} : S \subseteq [M]\}$ be the bundle values i can attain and set

$$d_i = \min\{|x - y| : x, y \in W_i, x \neq y\} > 0, \quad \varepsilon_i = \frac{d_i}{2M}.$$

(Since i has a positive weight, W_i is a finite set with at least two distinct elements — for instance 0 and $\max_g w_{i,g} > 0$ — so the minimum defining d_i ranges over a finite, non-empty set of positive gaps and is itself positive.) For each good with $w_{i,g} > 0$ the values 0 and $w_{i,g}$ lie in W_i , hence $d_i \leq w_{i,g}$ and so $\varepsilon_i < d_i \leq w_{i,g}$: ε_i is below every positive weight of i .

Let $\hat{w}$ replace each zero entry $w_{i,g} = 0$ by ε_i and keep positive entries. Then $\hat{w}$ is strictly positive, so by hypothesis it admits a full EFX allocation X ; we show the same X is EFX_0 for w . Consider a pair (i, j) with $i \neq j$ and let $g^* = \arg \min_{g \in X_j} w_{i,g}$; it suffices to check $v_i(X_i) \geq v_i(X_j \setminus \{g^*\})$ under w . As ε_i is below every positive weight, g^* also minimises $\hat{w}_{i,\cdot}$ over X_j : if X_j holds a good i values at 0 that good minimises under both w and $\hat{w}$, and otherwise the smallest positive good does. Write $L = v_i(X_i)$, $R = v_i(X_j \setminus \{g^*\})$ (values under w) and let a, b count the goods that i values at 0 in X_i and in $X_j \setminus \{g^*\}$. The EFX inequality for $\hat{w}$ at g^* is $L + a\varepsilon_i \geq R + b\varepsilon_i$, i.e. $(a - b)\varepsilon_i \geq R - L$. If EFX_0 under w failed then $L < R$, so $R - L > 0$ with $L, R \in W_i$ distinct and hence $R - L \geq d_i$; but $a - b \leq a \leq M$ gives $(a - b)\varepsilon_i \leq M\varepsilon_i = d_i/2 < d_i$, a contradiction. Thus $L \geq R$, the pair satisfies EFX_0 under w , and X is EFX_0 for w . $\square$

Corollary 1. *For each $(N, M) \in \{(4, 6), (4, 7), (4, 8)\}$, every non-negative additive profile admits a full EFX_0 allocation in which every agent receives at least one good.*

Corollary 2 (The three guarantees coincide). *Fix N, M and consider full allocations over the ordered partitions of $[M]$ into N non-empty bundles. The following are equivalent:*

- (P) *every strictly positive additive profile admits a full EFX allocation;*
- (Q) *every non-negative additive profile admits a full EFX_0 allocation;*
- (R) *every non-negative additive profile admits a full EFX_+ allocation.*

Proof. (P) $\Rightarrow$ (Q) is Theorem 2. (Q) $\Rightarrow$ (R) is immediate: on any profile the EFX_+ constraints (removing only positively-valued goods) form a subset of the EFX_0 constraints (removing any good), so every EFX_0 allocation is EFX_+ and a full EFX_0 allocation is in particular a full EFX_+ one. For (R) $\Rightarrow$ (P), let $w > 0$ be strictly positive. It is non-negative, so by (R) it admits a full EFX_+ allocation X over the ordered partitions; since every entry of w is positive, no good is valued at zero, so on w the EFX_+ , EFX_0 and plain EFX conditions coincide. Hence X is a full EFX allocation and (P) holds. $\square$

7 Related work and novelty

Three lines of work are especially close.

Mahara [8]. Mahara proved that an EFX allocation always exists when $M \leq N + 3$, even for general monotone valuations. For $N = 4$ this covers $M \leq 7$, so the (4, 6) and (4, 7) rows of Table 2 were already known analytically; we run them only as benchmarks. The first size his theorem does not reach for four agents is exactly $M = N + 4 = 8$, which is the size we settle here (for additive valuations).

Berger, Cohen, Feldman and Fiat [3]. They proved that four additive agents always admit an EFX allocation *leaving at most one good unallocated*, and more generally that N additive agents admit one leaving at most $N - 2$ goods unallocated. This is a theorem for *all* M , but it is an “almost full” guarantee: whether a *complete* four-agent EFX allocation always exists is left open. Our result removes the leftover item for the specific size $M = 8$.

Akrami, Mayorov, Mehlhorn, Srinivas and Weidenbach [1]. This recent work uses the same reduction idea — encode the negation of EFX existence and let **sat/unsat** decide — but in a different parameter regime. Their computations are for *three* agents ($M \in \{6, 7, 8\}$) under *monotone* valuations: they certify existence for three agents and seven goods, and find by SAT a submodular (hence monotone) counterexample for three agents and eight goods; the counterexample for general $n \geq 3$, $m \geq n + 5$ is then obtained from this seed by an explicit construction rather than by solving. Their Z3 SMT-over-LRA encoding did not terminate within 40 hours for three agents and seven goods (three agents and six goods took about 25 seconds). They explicitly leave the *additive* case open, and their computational results do not reach four agents.

Work	Agents	Goods	Valuations	Guarantee	Method
Plaut–Roughgarden [10]	2	any	monotone	full EFX exists	proof
Chaudhury et al. [6]	3	any	additive	full EFX exists	proof
Mahara [8]	any	$\leq n+3$	monotone	full EFX exists	proof
Berger et al. [3]	4	any	additive	<i>almost</i> full (≤ 1 leftover)	proof
Akrami et al. [1]	3	7	monotone	full EFX exists	SAT
Akrami et al. [1]	≥ 3	$\geq n+5$	submodular	<i>counterexample</i>	SAT + constr.
This report	4	8	additive	full EFX exists	SMT (LRA)

Table 3: The four-agent / eight-good additive cell is, to our knowledge, established here for the first time.

Novelty. Table 3 summarises these results. To the best of our knowledge, full EFX existence for four agents and eight goods under additive valuations has not been established before, by any method. The two works above are related but address different questions: Berger et al. [3] give an analytic “almost full” guarantee for all sizes but not a complete allocation, while Akrami et al. [1] compute with monotone valuations for three agents and leave the additive case open. The present result concerns additive valuations, four agents, and a complete allocation, at the fixed size $M = 8$.

Our (4, 8) instance sits exactly one good *below* the submodular counterexample threshold $M = N + 5 = 9$ of [1]: at (4, 9) a submodular (hence monotone) profile with no EFX allocation provably exists, whereas the additive case there is open. The natural next case is thus (4, 9): an additive EFX result there would show that additivity, not just monotonicity, is what makes EFX attainable at this size. We return to it in Section 10.

8 Discussion and limitations

No machine-checked certificate. The (4, 8) result rests on Z3 returning `unsat`, not on an externally replayable proof object. We attempted to extract one, but enabling Z3’s proof generation did not yield a usable proof: the solver returned `unknown` under the proof-producing configuration, so no DRAT- or Lean-checkable certificate is available. By contrast, Akrami et al. [1] verified their SAT encoding in Lean and their unsatisfiability proofs with DRAT-TRIM. Obtaining a checkable certificate for the (4, 8) query would require a different, proof-producing pipeline (for instance an exact-arithmetic LP/Farkas back-end), and remains open.

What underpins the trust in the result. Two things support Theorem 1. (i) The reduction itself is elementary and is argued in full in Section 3, with the one non-obvious step — restricting to non-empty partitions — proved sound in Proposition 1; the exactness of LRA reasoning (Section 3.2) means the `unsat` verdict is an exact impossibility over the reals, not a numerical artefact. (ii) The decision is reproducible: the query is exported as a standalone SMT-LIB 2 script and can be replayed by any LRA-capable solver, so the result does not depend on our particular driver code.

Scope of the claim. Theorem 1 covers all *non-negative additive* valuations at the *fixed* size $M = 8$. It says nothing about non-additive valuations, about chores (negative valuations), or about $M = 9$ or larger — each size is a separate, much larger computation we have not run, and existence at one size does not imply it at the next. The $\mathcal{P}$ -restriction to non-empty bundles is sound for the existence direction by Proposition 1, but means the same code, used to *find* a counterexample, would have to be extended to allow empty bundles before a `sat` answer could be trusted as a valid counterexample.

Scaling. The jump from 1,560 to 40,824 partitions ($6 \rightarrow 8$ goods) took the runtime from seconds to 62.5 hours. Each added good both multiplies the partition count and widens every constraint, so $M = 9$ is well out of reach of the present encoding without substantially stronger symmetry breaking or a fundamentally different decomposition.

9 Repository structure and running the code

The public repository is

<https://github.com/MAKMED1337/EFX>.

The main files are as follows. `main.cpp` is the Z3-based prover described in Section 4; `common.cpp` enumerates ordered non-empty partitions; `run-cpp.sh` compiles and runs a C++ source file; `main.py` is an earlier Python/Z3 prototype; `tex/` contains this report and bibliography; `proofs/` contains retained machine-verification artifacts; `logs/` contains retained solver logs; and `experiments/` contains historical experiment scripts and outputs.

To run the main prover, use

```
1 ./run-cpp.sh main.cpp 4 8
```

The two numeric program arguments are N and M : the number of agents and goods in the fixed instance to test. The current implementation enumerates ordered partitions into non-empty bundles, so it requires $M \geq N$. The program writes the assembled SMT-LIB 2 query to `proofs/problem_N_M.smt2` and then calls Z3. A `sat` result prints a valuation matrix witnessing a counterexample; an `unsat` result means no such counterexample exists for the encoded instance size; and `unknown` is inconclusive. With proof generation enabled at compile time, successful proof extraction writes a corresponding `proofs/proof_N_M.z3` file.

10 Conclusion and future work

We reduced the existence of full EFX allocations, for a fixed instance size, to a single satisfiability query over linear real arithmetic, and used it to show by computer that every additive profile with four agents and eight goods admits a full EFX allocation (Theorem 1). To our knowledge this is the first verification for the four-agent / eight-good additive case; it strengthens the “almost full” four-agent guarantee of [3] at this size and complements the recent SAT-based work of [1], which addressed three agents and left additive valuations open.

Several directions follow naturally:

- **A checkable certificate.** Our attempt to obtain a replayable proof failed — Z3 returned `unknown` under proof generation (Section 8). A proof-producing pipeline, for instance an exact LP/Farkas back-end emitting a certificate per allocation, would remove the reliance on trusting the solver.
- **The separating case $M = 9$.** For $N = 4$ the smallest submodular (hence monotone) EFX counterexample *guaranteed* by [1] occurs at $M = N + 5 = 9$: a submodular profile with four agents and *nine* goods provably admits *no* EFX allocation. Settling the *additive* case at $(4, 9)$ is therefore the natural next target, and it is informative either way. If the solver returns `unsat`, then additive EFX exists where monotone EFX fails — a *separation* showing that EFX existence depends on additivity, in the positive direction of the open question raised by [1]. If instead it returns `sat`, the witness is an additive instance with no EFX allocation — a refutation of EFX existence for additive valuations, i.e. a resolution of the main open problem for additive valuations. The obstacle is scale: $(4, 9)$ has $4! \binom{9}{4} = 186,480$ ordered partitions ($\approx 4.6\times$ the 40,824 of $(4, 8)$), an extra variable per agent, and longer constraints, so it will require parallelising or sharding the partition set and symmetry breaking stronger than sorting a single agent.
- **Variants.** Adapt the encoding to related notions: EFX for chores (negative valuations), or the relaxations (tEFX, and EF1 together with EEFX) that [1] study for three agents.

References

- [1] Hannaneh Akrami, Alexander Mayorov, Kurt Mehlhorn, Shreyas Srinivas, and Christoph Weidenbach. *A Counterexample to EFX: $n \geq 3$ Agents, $m \geq n + 5$ Items, Submodular Valuations via SAT-Solving*. arXiv preprint arXiv:2604.18216. 2026. arXiv: 2604.18216 [cs.GT].
- [2] Georgios Amanatidis, Haris Aziz, Georgios Birmpas, Aris Filos-Ratsikas, Bo Li, Hervé Moulin, Alexandros A. Voudouris, and Xiaowei Wu. “Fair Division of Indivisible Goods: Recent Progress and Open Questions”. In: *Artificial Intelligence* 322 (2023), p. 103965. DOI: 10.1016/j.artint.2023.103965.
- [3] Ben Berger, Avi Cohen, Michal Feldman, and Amos Fiat. *(Almost Full) EFX Exists for Four Agents (and Beyond)*. arXiv preprint arXiv:2102.10654. A short version appeared in Proc. AAAI 2022, vol. 36, no. 5, pp. 4826–4833. 2021. arXiv: 2102.10654 [cs.GT].
- [4] Eric Budish. “The Combinatorial Assignment Problem: Approximate Competitive Equilibrium from Equal Incomes”. In: *Journal of Political Economy* 119.6 (2011), pp. 1061–1103. DOI: 10.1086/664613.
- [5] Ioannis Caragiannis, David Kurokawa, Hervé Moulin, Ariel D. Procaccia, Nisarg Shah, and Junxing Wang. “The Unreasonable Fairness of Maximum Nash Welfare”. In: *ACM Transactions on Economics and Computation* 7.3 (2019), 12:1–12:32. DOI: 10.1145/3355902.

- [6] Bhaskar Ray Chaudhury, Jugal Garg, and Kurt Mehlhorn. “EFX Exists for Three Agents”. In: *Journal of the ACM* 71.1 (2024), 4:1–4:27. DOI: 10.1145/3616009.
- [7] Richard J. Lipton, Evangelos Markakis, Elchanan Mossel, and Amin Saberi. “On Approximately Fair Allocations of Indivisible Goods”. In: *Proceedings of the 5th ACM Conference on Electronic Commerce (EC)*. ACM, 2004, pp. 125–131. DOI: 10.1145/988772.988792.
- [8] Ryoga Mahara. “Extension of Additive Valuations to General Valuations on the Existence of EFX”. In: *Mathematics of Operations Research* 49.2 (2024). A preliminary version appeared in Proc. 29th Annual European Symposium on Algorithms (ESA) 2021, LIPIcs vol. 204, 66:1–66:15; arXiv:2107.09901, pp. 1263–1277. DOI: 10.1287/moor.2022.0044.
- [9] Leonardo de Moura and Nikolaj Bjørner. “Z3: An Efficient SMT Solver”. In: *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*. Vol. 4963. LNCS. Springer, 2008, pp. 337–340. DOI: 10.1007/978-3-540-78800-3_24.
- [10] Benjamin Plaut and Tim Roughgarden. “Almost Envy-Freeness with General Valuations”. In: *SIAM Journal on Discrete Mathematics* 34.2 (2020), pp. 1039–1068. DOI: 10.1137/19M124397X.

A Additional material

The prose of this report is licensed under Creative Commons Attribution-ShareAlike 4.0 International (CC BY-SA 4.0). Source-code listings included from repository files keep the license stated in those source files.

Listing 4: Full source of `main.cpp`.

```
1 // Copyright (c) 2026 Eduard Tykhoniuk.
2 // SPDX-License-Identifier: GPL-3.0-or-later
3
4 #include "common.cpp"
5 #include <algorithm>
6 #include <cstdlib>
7 #include <fstream>
8 #include <iostream>
9 #include <ostream>
10 #include <sstream>
11 #include <string>
12 #include <thread>
13 #include <vector>
14 #include <z3++.h>
15
16 const bool USE_SYMMETRY = true; // goods-relabelling symmetry break; set false to verify
    unsorted
17
18 // Encode the envy-freeness condition for a given partition.
19 // Returns a Z3 Boolean expression that is true iff the partition is envy-free
20 // under the current weight variables  $w[i][j]$  (agent  $i$ 's valuation for item  $j$ ).
21 z3::expr nonenvy(z3::context &ctx, int N, const std::vector<std::vector<int>> &partition,
22               const std::vector<std::vector<z3::expr>> &w) {
23     z3::expr_vector constraints(ctx);
24     for (int i = 0; i < N; ++i) {
25         // Total value of agent  $i$ 's own bundle.
26         z3::expr total = ctx.real_val(0);
27         for (int j : partition[i]) {
28             total = total + w[i][j];
29         }
30
31         for (int j = 0; j < N; ++j) {
32             if (i == j) {
33                 continue;
34             }
35
36             if (partition[j].size() <= 1) {
37                 continue;
38             }
39
40             // Total value of agent  $j$ 's bundle according to  $i$ 's valuations.
41             z3::expr other = ctx.real_val(0);
42             for (int k : partition[j]) {
43                 other = other + w[i][k];
44             }
45
46             // For every item  $k$  in  $j$ 's bundle, agent  $i$  should not envy  $j$ 
47             // after removing that item from  $j$ 's bundle.
48             for (int k : partition[j]) {
49                 constraints.push_back(total >= other - w[i][k]);
50             }
51         }
52     }
53     return z3::mk_and(constraints);
54 }
55
```

```

56 struct Variable {
57     std::string name;
58     std::vector<int> indices;
59 };
60
61 Variable parse_variable_name(std::string const &name) {
62     std::istringstream ss(name);
63     std::vector<std::string> tokens;
64     std::string token;
65     while (getline(ss, token, '_')) {
66         tokens.push_back(token);
67     }
68
69     std::vector<int> indices;
70     for (int i = 1; i < tokens.size(); i++) {
71         indices.push_back(stoi(tokens[i]));
72     }
73     return Variable{tokens[0], indices};
74 }
75
76 void print_frac(z3::expr f) { std::cout << f.numerator() << "/" << f.denominator() << " "; }
77
78 void print(z3::model m, int N, int M) {
79     using namespace std;
80     using namespace z3;
81     vector<vector<expr>> values(N, vector<expr>(M, expr(m.ctx())));
82     for (unsigned i = 0; i < m.size(); ++i) {
83         func_decl v = m[i];
84         auto name = v.name().str();
85         auto var = parse_variable_name(name);
86         auto value = m.get_const_interp(v);
87
88         if (var.name == "w") {
89             assert(var.indices.size() == 2);
90             values[var.indices[0]][var.indices[1]] = value;
91         } else {
92             assert(false);
93         }
94     }
95
96     for (int ind = 0; ind < values.size(); ind++) {
97         auto i = values[ind];
98         for (auto j : i) {
99             print_frac(j);
100         }
101         cout << "\n";
102     }
103 }
104
105 z3::solver get_solver(z3::context &ctx) {
106 #ifdef PROOF
107     z3::tactic simplify = z3::tactic(ctx, "simplify");
108     z3::tactic solve_eqs = z3::tactic(ctx, "solve-eqs");
109     z3::tactic blast_ite = z3::tactic(ctx, "blast-term-ite");
110     z3::tactic propagate = z3::tactic(ctx, "propagate-values");
111     z3::tactic smtt = z3::tactic(ctx, "smt");
112
113     // Combine them into a sequence
114     z3::tactic t = simplify & solve_eqs & blast_ite & propagate & smtt;
115     return t.mk_solver();
116 #else
117     return z3::solver(ctx);
118 #endif

```

```

119 }
120
121 int main(int argc, char **argv) {
122     if (argc != 3) {
123         std::cerr << "Usage: " << argv[0] << " N M\n"
124             << "   N: number of agents\n"
125             << "   M: number of goods\n";
126         return 2;
127     }
128
129     int N = std::atoi(argv[1]);
130     int M = std::atoi(argv[2]);
131     if (N <= 0 || M <= 0) {
132         std::cerr << "N and M must be positive integers.\n";
133         return 2;
134     }
135     if (M < N) {
136         std::cerr << "This prover enumerates non-empty bundles, so it requires M >= N.\n";
137         return 2;
138     }
139
140 #ifdef PROOF
141     // Enable proof generation globally.
142     std::cout << "Proof generation ENABLED" << std::endl;
143     z3::set_param("proof", "true");
144 #else
145     std::cout << "Proof generation DISABLED" << std::endl;
146 #endif
147
148     z3::context ctx;
149     z3::solver solver = get_solver(ctx);
150
151     std::cout << "N=" << N << " M=" << M << std::endl;
152     unsigned threads = std::max(1u, std::thread::hardware_concurrency());
153     std::cout << "Available hardware threads: " << threads << "\n";
154
155     // Create weight variables.
156     std::vector<std::vector<z3::expr>> w(N, std::vector<z3::expr>(M, z3::expr(ctx)));
157     for (int i = 0; i < N; ++i) {
158         for (int j = 0; j < M; ++j) {
159             std::string name = "w_" + std::to_string(i) + "_" + std::to_string(j);
160             w[i][j] = ctx.real_const(name.c_str());
161         }
162     }
163
164     // Positivity.
165     for (int i = 0; i < N; ++i) {
166         for (int j = 0; j < M; ++j) {
167             solver.add(w[i][j] > 0);
168         }
169     }
170
171     // First agent's weights sorted (goods-relabelling symmetry break).
172     // Set USE_SYMMETRY=false to disable it (much slower; for cross-checking).
173     if (USE_SYMMETRY) {
174         for (int j = 0; j < M - 1; ++j) {
175             solver.add(w[0][j] <= w[0][j + 1]);
176         }
177     }
178
179     // Generate partitions.
180     auto partitions = ordered_groups(N, M);
181     std::cout << "Number of partitions = " << partitions.size() << std::endl;

```

```

182
183 // Build the disjunction.
184 z3::expr_vector valid_exprs(ctx);
185 for (const auto &part : partitions) {
186     valid_exprs.push_back(nonenvy(ctx, N, part, w));
187 }
188 z3::expr or_valid = z3::mk_or(valid_exprs);
189 solver.add(!or_valid);
190
191 // Export SMT-LIB2 for external verification.
192 std::string smt_name = "proofs/problem_" + std::to_string(N) + "_" + std::to_string(M) + ".
    smt2";
193 std::ofstream smt(smt_name);
194 smt << solver.to_smt2() << std::endl;
195 smt.close();
196 std::cout << "SMT-LIB2 script saved to " << smt_name << std::endl;
197
198 std::cout << "Solving..." << std::endl;
199 z3::check_result result = solver.check();
200
201 if (result == z3::unsat) {
202     std::cout << "No counterexample :(" << std::endl;
203     // Attempt to get proof (may throw if Z3 doesn't support it for LRA).
204 #ifdef PROOF
205     z3::expr proof = solver.proof();
206     std::string proof_name = "proofs/proof_" + std::to_string(N) + "_" + std::to_string(M) +
        ".z3";
207     std::ofstream proof_file(proof_name);
208     proof_file << proof << std::endl;
209     proof_file.close();
210     std::cout << "Proof saved to " << proof_name << std::endl;
211 #endif
212 } else if (result == z3::sat) {
213     std::cout << "Counterexample found:" << std::endl;
214     z3::model m = solver.get_model();
215     std::cout << m << std::endl;
216     print(m, N, M);
217 } else {
218     std::cout << "Unknown result:" << result << std::endl;
219 }
220
221 return 0;
222 }

```